

Security Vulnerability Assessment Report

Target Application: OWASP Juice Shop

Author: Moe Htet Ar Kar (Phoe Cho)

Scope: Authentication, Input Validation, Access Control, and Token Security

Executive Summary

This report documents the security assessment and vulnerability testing conducted against the OWASP Juice Shop application. Multiple critical and high-risk vulnerabilities were identified, successfully demonstrated within the authorized assessment environment, and analyzed to determine their root causes, security impact, and appropriate remediation strategies.

The assessment focused primarily on authentication mechanisms, input validation, access control enforcement, and token security. The identified vulnerabilities demonstrate weaknesses that could potentially lead to unauthorized authentication, client-side code execution, unauthorized access to other users' resources, and privilege escalation.

Detailed Findings

1. SQL Injection (SQLi) in Authentication

Vulnerability Type

SQL Injection (SQLi)

Severity

Critical

Description

The application's authentication mechanism failed to properly validate and parameterize user-supplied input within the login form. As a result, attacker-controlled input could alter the logic of the backend SQL query.

Exploitation

Input Supplied:

- **Email:** ' OR 1=1--
- **Password:** 123

Result:

The injected payload altered the backend SQL query logic and caused the authentication condition to evaluate as **TRUE**. This resulted in unauthorized authentication and returned the administrator account:

- **Account:** `admin@juice-sh.op`
- **User ID:** `1`

The successful authentication provided access to privileged application functionality.

Security Impact

Successful exploitation of this vulnerability could allow an unauthenticated attacker to bypass the application's authentication mechanism and gain unauthorized access to user or administrative accounts.

Remediation

- Implement **parameterized queries / prepared statements** for all database interactions.
- Ensure user-supplied input is always treated as data rather than executable SQL syntax.
- Apply server-side input validation as an additional security control.
- Avoid dynamically constructing SQL statements using untrusted user input.

2. DOM-Based Cross-Site Scripting (XSS)

Vulnerability Type

Cross-Site Scripting (XSS) / DOM-Based XSS

Severity

Medium / High

Description

The application demonstrated susceptibility to DOM-based Cross-Site Scripting when user-controlled input reached an unsafe DOM sink. Although standard inline `<script>` elements were escaped by the frontend framework, the application's client-side processing still allowed attacker-controlled input to reach a security-sensitive DOM context.

Exploitation

Input Supplied via Search Bar:

```
<iframe src="javascript:alert(`xss`)">
```

Result:

The payload was processed by the application's client-side DOM logic and triggered a JavaScript alert dialog, confirming that attacker-controlled input could reach an executable DOM context.

Security Impact

Successful exploitation of DOM-based XSS could allow an attacker to execute arbitrary JavaScript within the security context of a victim's browser, potentially enabling unauthorized actions, session-related attacks, or manipulation of application content.

Remediation

- Apply **context-aware output encoding** before rendering user-controlled data into the DOM.
 - Avoid unsafe DOM sinks such as `innerHTML` when processing untrusted data.
 - Prefer safe DOM APIs that treat input as text rather than executable markup.
 - Enforce a strict **Content Security Policy (CSP)** to reduce the impact of potential XSS vulnerabilities.
 - Validate and sanitize client-side input before processing or rendering it.
-

3. Insecure Direct Object References (IDOR)

Vulnerability Type

Insecure Direct Object References (IDOR)

Severity

High

Description

The application failed to properly verify whether the authenticated user was authorized to access the resource identified by a user-controlled object identifier in an API request.

Exploitation

HTTP traffic was intercepted using **Burp Suite**, and the basket endpoint was modified from:

```
/rest/basket/6
```

to reference another user's resource:

```
UserId: 25
```

Result:

The server returned sensitive shopping-basket information belonging to another user without enforcing appropriate server-side authorization checks.

Security Impact

Successful exploitation could allow an authenticated attacker to access resources belonging to other users, resulting in unauthorized data disclosure and horizontal privilege escalation.

Remediation

- Implement robust **server-side authorization checks** for every object-level request.
 - Verify that the authenticated user owns or has explicit permission to access the requested resource.
 - Do not rely on client-controlled identifiers as evidence of authorization.
 - Consider using unpredictable identifiers, such as **UUIDs**, where appropriate.
 - Implement centralized object-level authorization controls across API endpoints.
-

4. JWT Privilege Escalation & Inadequate Signature Validation

Vulnerability Type

Broken Authentication / Insecure Token Handling

Severity

Critical

Description

The application did not adequately validate the cryptographic integrity of the JSON Web Token (JWT) before trusting authorization-related claims contained within its payload.

As a result, modification of the token's authorization-related claims could potentially allow an attacker to escalate privileges without possessing legitimate administrative credentials.

Exploitation

1. A valid JWT Bearer token was intercepted using **Burp Suite Proxy/Repeater**.
2. The JWT payload was decoded and the **role** parameter was modified from:

```
"role": "customer"
```

to:

```
"role": "admin"
```

3. The modified token was supplied through the **Authorization** header.
4. The browser's **localStorage token** parameter was also updated with the modified token.

Result:

The server accepted the modified token and returned a:

```
200 OK
```

response.

The modified authorization claim was accepted, allowing access to the restricted administrative route:

```
/#/administration
```

Security Impact

Successful exploitation could allow an attacker to bypass authorization controls and escalate privileges from a standard user account to an administrative role.

This could potentially provide access to sensitive administrative functionality and data.

Remediation

- Enforce strict **cryptographic signature verification** for every incoming JWT.
- Validate the JWT signature using an appropriately protected symmetric secret or an asymmetric public/private key pair.
- Validate critical JWT claims such as **issuer (iss)**, **audience (aud)**, expiration (**exp**), and other application-specific claims where applicable.
- Do not trust client-modifiable authorization claims without independently validating their integrity and authority.
- Consider retrieving authorization roles from a trusted server-side source when appropriate.
- Implement centralized authorization checks independently of client-side token contents.

Conclusion

The security assessment successfully demonstrated multiple vulnerabilities across the application's authentication, input validation, access control, and token-security layers.

The identified issues include:

1. **SQL Injection (SQLi)** affecting authentication.
2. **DOM-Based Cross-Site Scripting (XSS)** through unsafe DOM handling.
3. **Insecure Direct Object References (IDOR)** resulting in unauthorized access to another user's resources.
4. **JWT Privilege Escalation** caused by inadequate token-integrity validation.

These findings demonstrate the importance of implementing strong server-side security controls, including parameterized database queries, context-aware output encoding, object-level authorization, and rigorous cryptographic token validation.

Implementing the recommended remediation measures would substantially reduce the application's exposure to unauthorized access, data disclosure, client-side code execution, and privilege-escalation attacks.